System and Devices (SYS3)

Lecture Review – Week 6

Dr Pengcheng Liu

Welcome

Check-In Code:

708768

Log into checkin.york.ac.uk, find the session and enter the code.

Valid until Thursday, 19 March 2026 11:00

Week 6 Module
Overview and Updates

Module Overview and Updates

- **Week 6**:

Lecture: this week, our discussion was focused on dynamic scheduling and Tomasulo's Algorithm in Lectures 9 and 10, respectively.

Lab: In Week 6, we focused on data hazard (registers). Data hazards occur when the intended order of read and write operations is incorrectly altered, typically due to pipeline timing effects. This can occur in memory read/write sequences, and also in sequences of register reads and writes. Data Hazards involving memory can be quite complex to deal with and won't be covered in the practical sessions.

In this case we are concerned only with register hazards, and the order of the register accesses that can occur due to pipeline effects. This results in multiple hazard types, RAW, WAR, and WAW. We investigated how these occur and devise suitable solutions.

Exercises/Quizzes: Quiz 2 will be released on 20/03/2026. Covering content from Week 4 to Week 7 lectures. Responses will be automarked.

Module Overview and Updates

- **Next Week (Week 7)**:

Lecture: In Week 7, our discussion will be focused on some principles of speculative execution and superscalar processors in Lecture 11 and Lecture 12, respectively.

Lab: We will continue working on solutions for Week6's lab tasks.

Exercises/Quizzes: Solution of Exercise 2 will be released on 27/03/2026. Quiz 2 will be discussed in the Lecture Review session on 26/03/2026.

Questions on Padlet

Padlet Questions

L6 Pipeline Hazards– Stalls: *Can we add the stalls/bubbles after the problematic instruction has started execution, let's say two instructions are trying to access the same resource but the issue is not in IF but rather later in a later stage, until now we have always added the stall before the issue causing instruction, what other ways are there to resolve these types of issues?*

We can usually design mechanism to stall the problematic instruction, but there some other ways/techniques to reduce stalls and maximize instruction throughput:

In **out-of-order execution** architectures, instructions are **dynamically reordered** to maximize ILP and minimize stalls due to dependencies. The processor's hardware **tracks data dependencies** and **executes** instructions as **soon as their operands are available**, even if they are not in program order. This allows instructions that do not depend on the problematic resource to proceed, while the dependent instructions are delayed until the resource is available.

Padlet Questions

L6 Pipeline Hazards– Stalls: *Can we add the stalls/bubbles after the problematic instruction has started execution, what other ways are there to resolve these types of issues?*

Instead of stalling the pipeline, the processor can detect data hazards and **forward the required resource** directly from the execution stage to the dependent instruction, **bypassing** the need to wait for the data to be written back to the register file. This technique reduces pipeline stalls by allowing instructions to proceed even when dependent data is not yet available in the register file.

The technique of **Hardware Interlocking** involves using hardware mechanisms to detect and resolve resource conflicts dynamically during instruction execution. For example, the processor can employ **reservation stations or scoreboard** mechanisms to manage resource access and avoid conflicts between instructions competing for the same resources. By dynamically scheduling instructions based on resource availability, hardware interlocking minimizes stalls and improves pipeline efficiency.

Padlet Questions

L6 Pipeline Hazards– Stalls: *Can we add the stalls/bubbles after the problematic instruction has started execution, what other ways are there to resolve these types of issues?*

Speculative execution involves executing instructions ahead of time based on **predictions about future control flow or data dependencies**. If the processor encounters a resource conflict during speculative execution, it can rollback the pipeline and discard the incorrect results without affecting program correctness.

Speculative execution helps **mitigate stalls** by allowing the processor to make progress on **independent instructions** while awaiting resolution of resource conflicts.

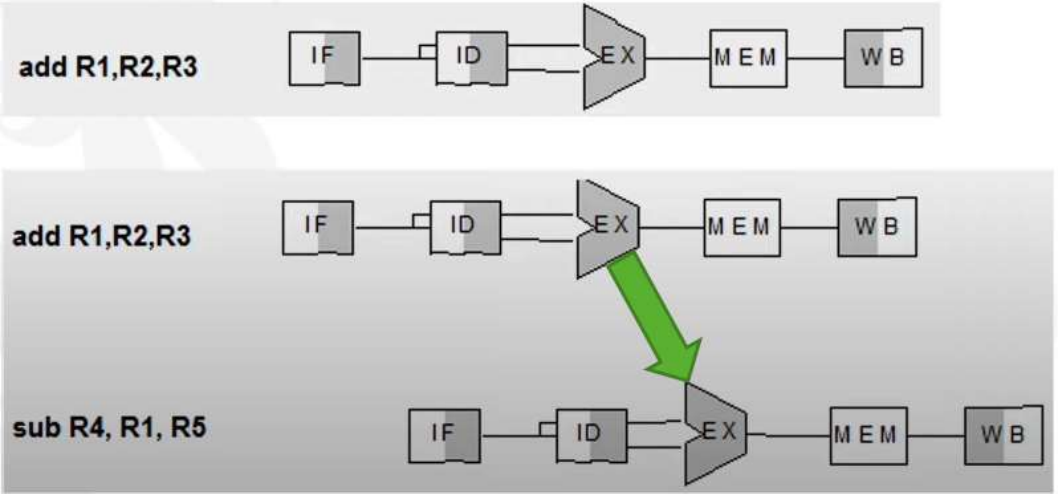
Padlet Questions

L6 Pipeline Hazards– Operand Forwarding: *I did not understand the data / operand forwarding. What does forward data if possible mean on slide 14?*

Data forwarding, i.e. data bypassing, is a technique used in pipelined processors to resolve data hazards without introducing pipeline stalls. In a 5-stage RISC instruction pipeline, data forwarding **enables instructions to access operands directly** from the pipeline stages where they are produced, rather than waiting for them to be written back to the register file.

How data forwarding works and how it resolves data hazards?

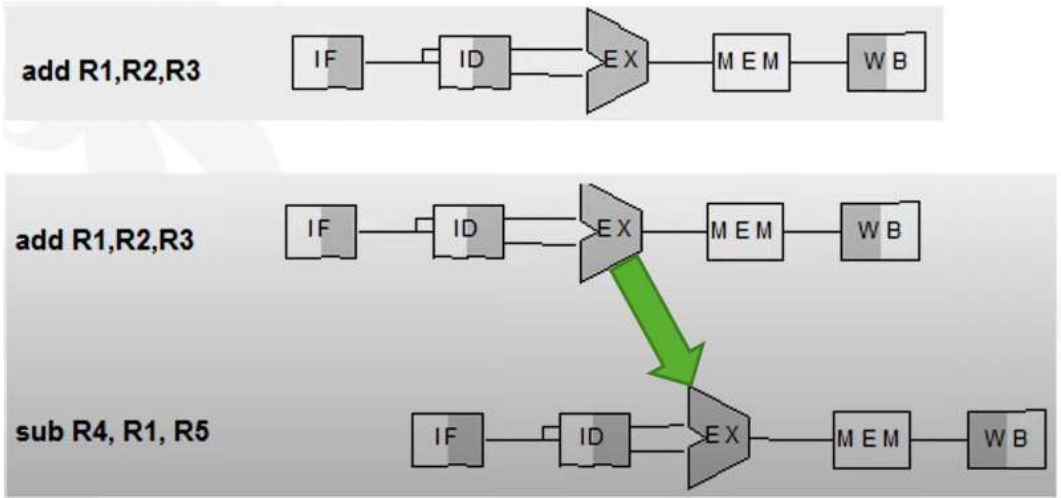
Data hazards occur when an instruction depends on the result of a previous instruction that has not yet completed execution. In a pipelined processor, data hazards can arise due to instructions in the pipeline that produce results needed by subsequent instructions.



Padlet Questions

L6 Pipeline Hazards– Operand Forwarding: *I did not understand the data / operand forwarding. What does forward data if possible mean on slide 14?*

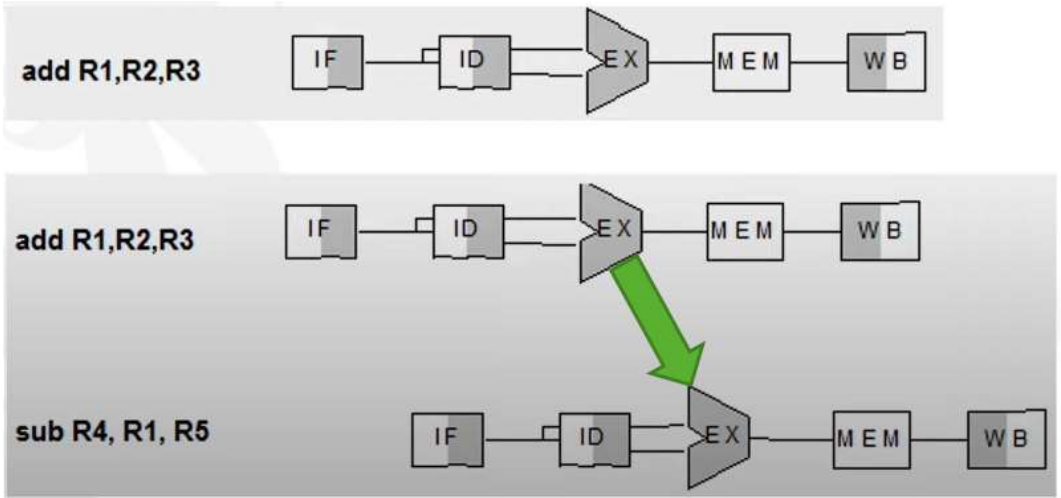
When an instruction in the execution stage (EX) produces a result that is needed by a subsequent instruction in the pipeline, the processor **forwards the result directly from the execution stage to the stage where it is needed**. For example, if an instruction in the execute stage computes a value that is needed by an instruction in the decode stage (ID), the processor forwards the computed value directly to the decode stage.



Padlet Questions

L6 Pipeline Hazards– Operand Forwarding: *I did not understand the data / operand forwarding. What does forward data if possible mean on slide 14?*

Data forwarding bypasses the register file and allows instructions to access operands directly from the pipeline stages where they are produced. By **bypassing the register file**, the processor **reduces the latency** associated with accessing operands and **avoids introducing pipeline stalls** to wait for operand availability.
This helps maintain pipeline throughput and improves overall processor performance by reducing the impact of data hazards on instruction execution.



Padlet Questions

L6 Pipeline Hazards– Operand Forwarding: *I did not understand the data / operand forwarding. What does forward data if possible mean on slide 14?*

Let's consider **an example** with three instructions:

Instruction 1 (ADD): Adds the values from registers $R1 and $R2 and stores the result in register $R3.
Instruction 2 (SUB): Subtracts the value from register $R3 from the value in register $R4 and stores the result in register $R5.
Instruction 3 (MUL): Multiplies the values from registers $R5 and $R6 and stores the result in register $R7.

Padlet Questions

L6 Pipeline Hazards– Operand Forwarding: *I did not understand the data / operand forwarding. What does forward data if possible mean on slide 14?*

```
     IF   ID   EX   MEM   WB
1:  ADD
2:       SUB
3:            MUL
```

Without data forwarding, the pipeline might look like above:

Instruction 1 (ADD): Moves through the pipeline stages.
Instruction 2 (SUB): Waits for the result of the ADD instruction to be written back to the register file before it can proceed.
Instruction 3 (MUL): Waits for the result of the SUB instruction to be written back to the register file before it can proceed.

Padlet Questions

***L6 Pipeline Hazards– Operand Forwarding**: I did not understand the data / operand forwarding. What does forward data if possible mean on slide 14?*

```
        IF   ID   EX   MEM   WB
1:  ADD        ADD
2:        SUB        SUB
3:             MUL
```

With data forwarding:
After the ADD instruction in the EX (Execute) stage, the result (sum of $R1 and $R2) is forwarded directly to the subsequent stages of the pipeline.

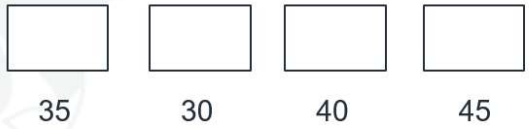
When the SUB instruction reaches the ID (Instruction Decode) stage, it detects that it needs the result from the ADD instruction. With data forwarding, the SUB instruction receives the forwarded result directly from the EX stage, bypassing the need to wait for the result to be written back to the register file.

Padlet Questions

Pipeline Design Question TL1 Instruction Pipeline & Performance: *Please could you re go over how you solve how long it will take for 100 instructions, I don't get why 1 instruction would take 200ns would it not be 35+30+40+45+5?*

Pipeline Design

The time delay of a 4-segment pipeline in different segments are t1=35ns, t2=30ns, t3=40ns, t4=45ns. The interface register delay time is t=5ns. How long would it take to complete 100 instructions in the pipeline? (Assume there is no dependency between the instructions).



50 × 100 = 5000ns

45 + 5 = 50 ns

First instruction requires 50 × 4 = 200ns to complete

So, in total: 200 + 99 × 50 = 5150ns to complete

As we have discussed in the lecture, the pipeline rate is **limited by the slowest pipeline stage**.

So in this case, the fourth stage is the dominating stage. So (45+5)=50ns is the clocking time for this pipeline. So 1-1 instruction gets completed every 50ns. As it is a 4-stage pipeline, so the first instruction will take 4*50= 200ns to complete.

Padlet Questions

L7 Control Hazards - filling branch delay slot:

Can you explain further Slide 8 and 9, i do not understand the diagrams. Also can you expand further on what is meant by a branch being taken or untaken?

A **delayed branch** is a technique used in certain processor architectures, particularly in RISC, to mitigate the performance impact of branch instructions on pipeline efficiency. When a delayed branch is encountered, the branch instruction is executed in the current cycle, but its effect on the program flow is deferred by one cycle. In other words, **the branch instruction is not fully resolved until the subsequent cycle**.

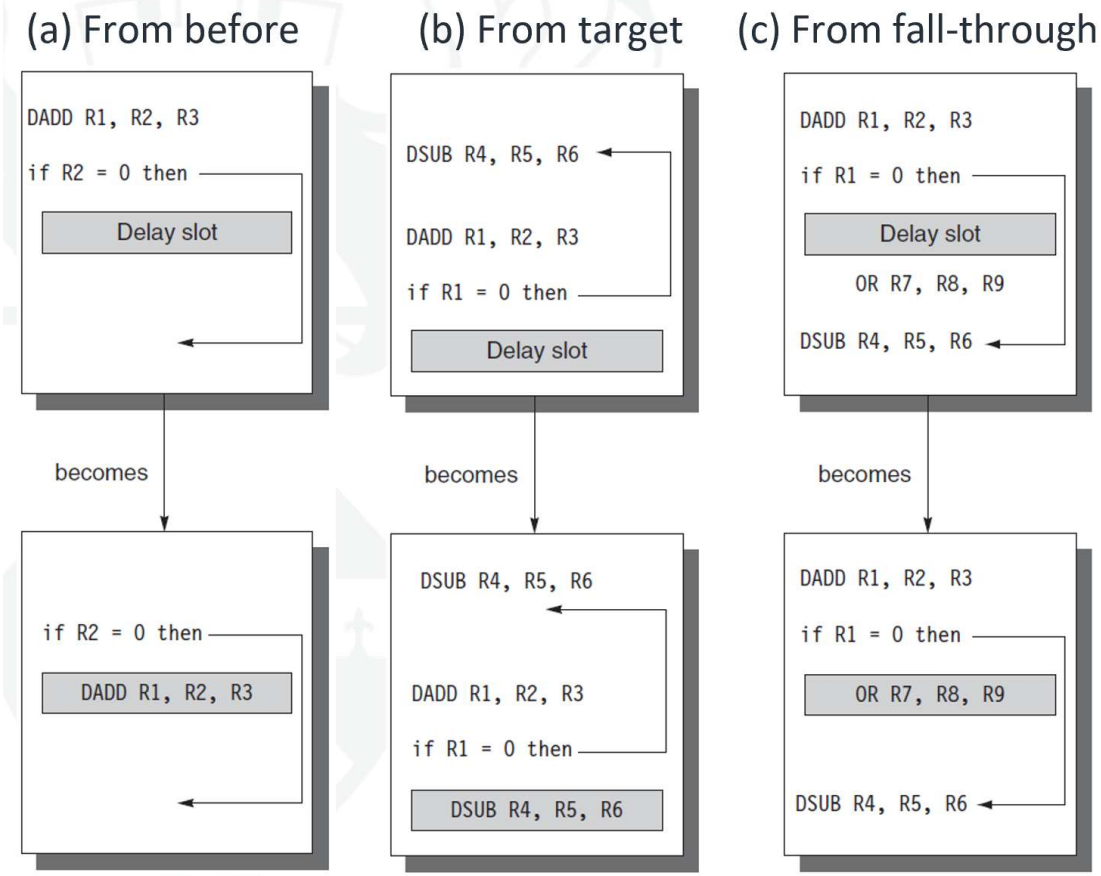
The purpose of a delayed branch is to **keep the pipeline full and maintain high throughput** by executing an instruction in the **branch delay slot**, which is the instruction immediately following the branch instruction. The instruction in the branch delay slot is executed regardless of whether the branch is taken or not taken. This helps to maximize the utilization of the pipeline and reduce stalls.

Padlet Questions

L7 Control Hazards - filling branch delay slot:

Can you explain further Slide 8 and 9, i do not understand the diagrams. Also can you expand further on what is meant by a branch being taken or untaken?

The three cases (**from before, from target, and from fall-through**) are three representative scenarios for filling the delay slot: **ideal (best)** choice - no dependency; branch is **taken with high possibility**; and branch is **not taken with high possibility**, respectively.

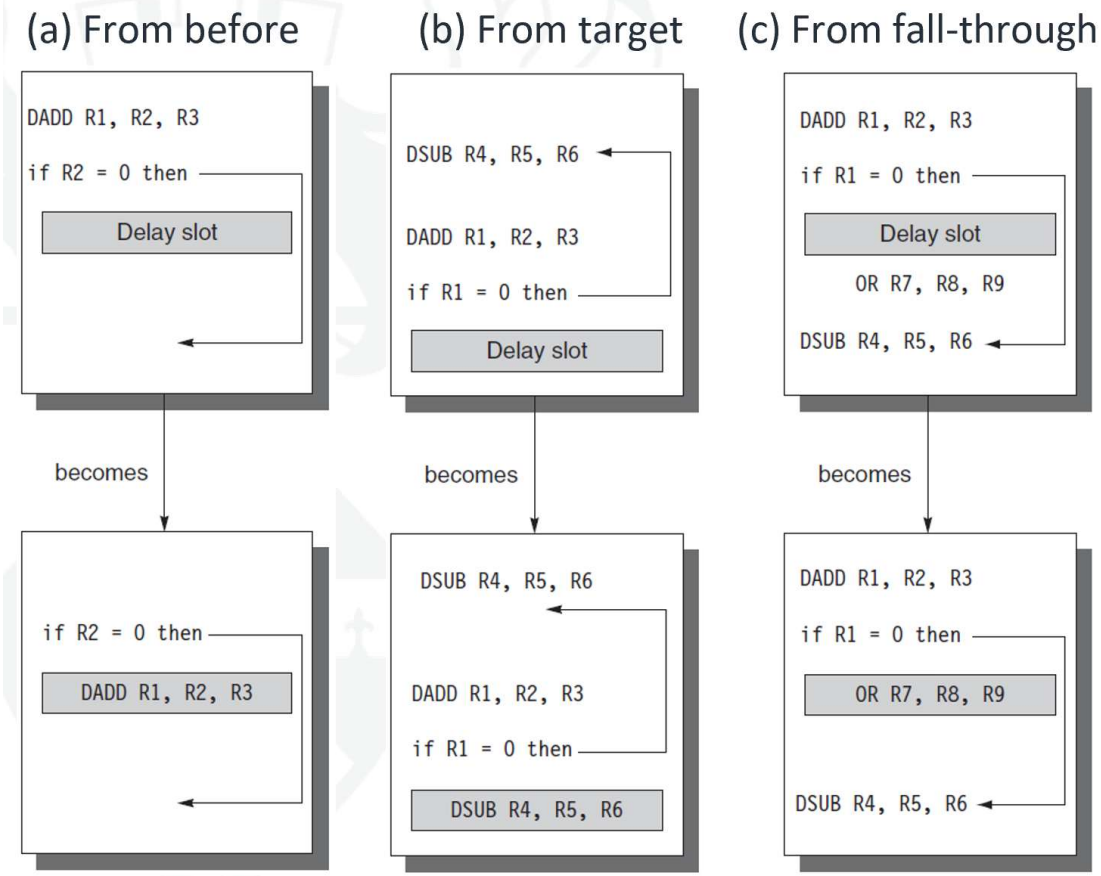


Padlet Questions

L7 Control Hazards - filling branch delay slot:

Can you explain further Slide 8 and 9, i do not understand the diagrams. Also can you expand further on what is meant by a branch being taken or untaken?

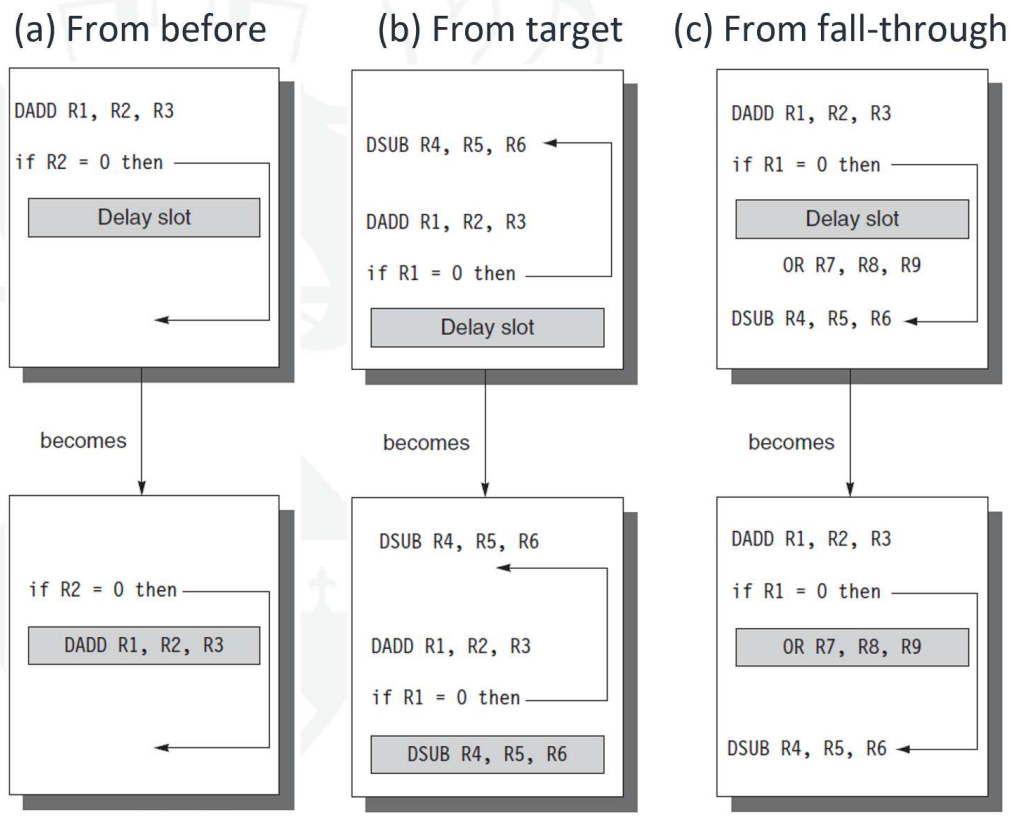
From before: the instruction in the branch delay slot is fetched from the instruction stream before encountering the branch instruction. The instruction is chosen based on its **independence** from the branch instruction and its ability to execute safely in the branch delay slot. This method is often used in hardware implementations where the **processor can fetch instructions ahead of time** to fill the delay slot.



Padlet Questions

L7 Control Hazards - filling branch delay slot:

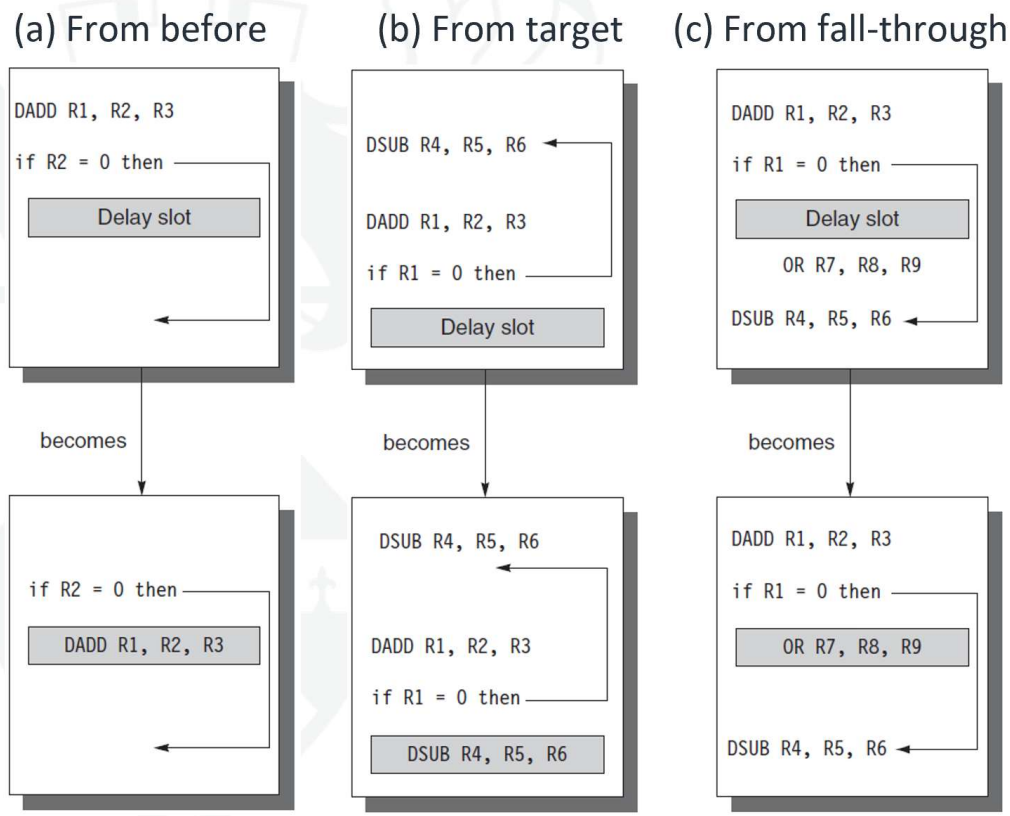
Filling branch delay slot **from target** is used for the scenario when the **branch is taken with high possibility**. Usually, the target instruction will be copied and put in the delay slot so it can be reached by another path. In the example on slide 9, DSUB will be surely executed if the branch (if R1=0 then) is taken. In this example, we considered that it is a scenario in which a branch is highly possible to be taken, so we filled the delay slot with the target instruction - DSUB. When executing, the DSUB will be executed, then we go through the loop, and the branch condition statement (if R1=0 then), and when this branch is in progress, the surely executed target instruction is fetched, then jump to the instruction immediately after DSUB. So the method of from target is usually used when a branch is highly likely to be taken.



Padlet Questions

L7 Control Hazards - filling branch delay slot:

Filling branch delay slot from **fall-through** is usually used for the case when **the branch is NOT taken with high possibility**. In the example on slide 9, OR is the instruction that will be executed when the branch (if R1=0 then) is not taken.
The control flow is, we have a DADD, then check the value of R1, if R1=0 then we go to DSUB, if R1 !=0 then we execute the OR instruction. In this example, we considered that it is a scenario in which a branch is highly possible NOT to be taken, in such case, the control will reach OR instruction. So we filled the delay slot with the OR instruction, meaning OR will always be executed whenever the branch is being fetched. So we usually use bring-from-fall-through method when there is a high possibility the branch is not taken.

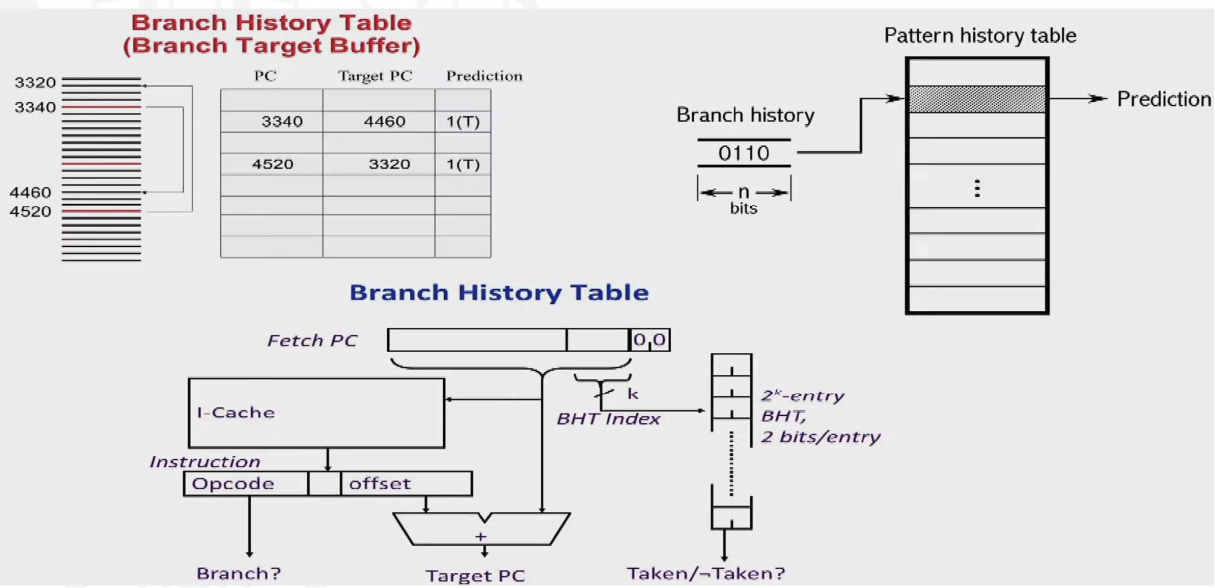


Padlet Questions

L7 Control Hazards - Branch Prediction Buffer (BPB) and Folding: *Slide 19-21, can you please go through an example and explain these concepts again?*

The Branch Prediction Buffer (BPB), also known as the Branch Prediction Table or Branch History Table, is a hardware structure used to predict the outcome of branch instructions before they are fully resolved.
The BPB **stores historical information** about previous branch instructions and their outcomes, allowing the processor to make informed predictions about future branches.

When a branch instruction is encountered, the BPB is consulted to predict whether the branch is likely to be taken or not taken. The predicted outcome of the branch is used to speculatively fetch instructions along the predicted path, keeping the pipeline full and minimizing stalls.



4K-entry BHT, 2 bits/entry, about 80-90% correct predictions

Padlet Questions

L7 Control Hazards - Branch Prediction Buffer and Folding:

Branch Prediction Buffer example:

Consider a simple scenario where we have a loop that iterates multiple times. The branch instruction determines whether to continue looping or exit the loop.
Let's say the branch prediction buffer maintains a history of previous branch outcomes for the loop. Here's a simplified representation of a branch prediction buffer:

Loop:

	IF	ID	EX	MEM	WB
1:	Load	Add	Sub	Branch	
2:	Load	Add	Sub	Branch	
3:	Load	Add	Sub	Branch	
4:	Load	Add	Sub	Branch	

Padlet Questions

L7 Control Hazards - Branch Prediction Buffer and Folding:

Branch Prediction Buffer example:

Loop:

	IF	ID	EX	MEM	WB
1:	Load	Add	Sub	Branch	
2:	Load	Add	Sub	Branch	
3:	Load	Add	Sub	Branch	
4:	Load	Add	Sub	Branch	

In this example, the branch instruction predicts whether the loop will continue (taken) or exit (not taken) based on the historical outcome stored in the branch prediction buffer.

The branch prediction buffer helps the processor make informed predictions about future branches, enabling speculative execution and minimizing pipeline stalls.

Padlet Questions

L7 Control Hazards - Branch Prediction Buffer and Folding:

Branch Folding

Branch folding is a technique to **optimise the execution of branch instructions**, particularly in pipelined processors. It involves **merging multiple branch instructions** into a single branch instruction to reduce the number of branches in the code and improve pipeline efficiency.
The goal of branch folding is to **minimise the overhead** associated with branch instructions and **maximise the utilisation** of the instruction pipeline. By reducing the number of branch instructions executed, branch folding can help improve processor throughput and reduce the impact of branches on program execution speed.
Branch folding can be especially useful in cases where **multiple branches occur in close proximity to each other and have similar conditions**. Instead of executing each branch instruction separately, they are merged or folded into a single instruction, streamlining the execution process.

Padlet Questions

UNIVERSITY of York

L7 Control Hazards - Branch Prediction Buffer and Folding:

Branch Folding:

Example:
Consider two consecutive conditional branch instructions in a loop that have similar conditions.

Branch folding could merge these two branches into a single branch instruction, reducing the number of branch instructions executed during each iteration of the loop.

Branch folding can be performed by hardware mechanisms in the processor or through compiler optimizations during the compilation process.

Padlet Questions

L7 Control Hazards - Branch Prediction Buffer and Folding:

Branch Folding example:

Consider a scenario where we have two consecutive conditional branch instructions in a loop, each determining whether to continue iterating or exit the loop.

Here's the original loop with two separate branch instructions:

Loop:

	IF	ID	EX	MEM	WB
1:	Load	Add	Sub	Branch1	
2:	Load	Add	Sub	Branch2	
3:	Load	Add	Sub	Branch1	
4:	Load	Add	Sub	Branch2	

Now, let's visualize the loop after applying branch folding to merge the two branch instructions into a single branch instruction:

Loop:

	IF	ID	EX	MEM	WB
1:	Load	Add	Sub	Branch	
2:	Load	Add	Sub	Branch	
3:	Load	Add	Sub	Branch	
4:	Load	Add	Sub	Branch	

Padlet Questions

L7 Control Hazards - Branch Prediction Buffer and Folding:

Branch Folding example:

In this example, branch folding reduces the number of branch instructions executed during each iteration of the loop, optimizing pipeline efficiency and improving processor throughput.

Branch folding can be performed by hardware mechanisms within the processor or through compiler optimizations during the compilation process.

Padlet Questions

***S2 Week 4 L7 - Control hazards 2**: Terminology Confusion*

We have now talked about a "Branch Target Buffer", a "Branch Target Cache", and a "Branch Prediction Buffer" (in the Lecture Review slides). Are these all the same thing?

BTB and BTC are synonyms.
BPB, BPT, and BHT are synonyms. But BTB and BPB aren't the same.

BTB also called Branch Target Cache BTC, is used for **storing predicted target addresses** of branch instructions.

BPB, also called BPT or BHT, is used for **storing predictions** about whether a branch will be **taken or not taken**, usually based on past behaviour, and it doesn't store target addresses, only the direction (taken or not taken).

Padlet Questions

***S2 Week 4 L7 - Control hazards 2**: Terminology Confusion*

We have now talked about a "Branch Target Buffer", a "Branch Target Cache", and a "Branch Prediction Buffer" (in the Lecture Review slides). Are these all the same thing?

Here is how they work together:

- BTB provides the target address if you predict the branch is taken.
- BPB/BHT/BPT provides the direction prediction (taken/not taken).
- If the branch is predicted taken, the BTB gives you where to fetch next.
- If predicted not taken, you proceed sequentially.

Padlet Questions

***S2 Week 4 L7 – Zero Penalty**: Review and* Clarification

The **"zero-penalty"** concept refers **to the case when the branch prediction is correct and** the branch target is **found in the Branch Target Buffer (BTB)**.
In this situation:

- The CPU can **immediately fetch** instructions from the predicted target **without delay**.
- There's **no stall**, and the pipeline flows seamlessly—**zero penalty** in this ideal scenario.

Penalty when prediction is wrong (misprediction)
When the prediction is **wrong**:

- The pipeline **must flush** the incorrectly fetched instructions.
- There's typically a **penalty of one cycle** to update the BTB and **another cycle**, if needed, to restart fetching from the correct address.

This does introduce a **penalty**, even if it's relatively small compared to not using prediction at all.

Padlet Questions

S2 Week 4 L7 - Control hazards 2: *What happens when a branch-target buffer is used and the prediction is wrong? The lecture says that there is zero penalty, but how can this be the case if the prediction is wrong and therefore the predicted PC is fetched in error? I've understood zero-penalty as meaning there is no delay after the branch if the prediction is wrong, is that correct? Thanks*

When a Branch Target Buffer is used and the prediction is wrong, the pipeline will have fetched the incorrect instruction based on the predicted Program Counter. However, the idea of **zero-penalty** in this context means that the pipeline does not stall before making a prediction—it immediately proceeds with the predicted PC, avoiding an initial delay.
Zero-penalty does not mean that mispredictions have no cost. It refers to the fact that BTB enables an **immediate instruction fetch** without waiting for branch resolution. If the prediction is correct, the branch is executed with zero delay. If the prediction is wrong, there is a **penalty of pipeline flush + refetch**, but it is still better than stalling upfront while waiting for the correct branch outcome.

Padlet Questions

S2 Week 5 L8 - Compiler techniques 1: Latency and Clock Cycles

Figure 3.2 of the textbook (copied on slide 8) lists the various latencies of different instructions, and the proceeding dependent instruction. Given that MIPS (used in the textbook) is a RISC architecture, where each instruction takes a fixed amount of clock cycles, how can different instructions have different latencies?

In a basic 5-stage MIPS RISC pipeline, each instruction typically completes one stage per clock cycle, and the pipeline ideally outputs one instruction per cycle after filling up.

This creates the illusion that all instructions take the same time to execute (uniform instruction timing), but in reality, **some instructions produce their results sooner or later relative to others**.

Instruction producing result	Instruction using result	Latency in clock cycles
FP ALU op	Another FP ALU op	3
FP ALU op	Store double	2
Load double	FP ALU op	1
Load double	Store double	0

Padlet Questions

UNIVERSITY of York

S2 Week 5 L8 - Compiler techniques 1: Latency and Clock Cycles

Why do different instructions have different latencies?

- **Latency** refers to how many cycles it takes from starting an instruction to getting its result. **Throughput** refers to how often you can start a new instruction. In a pipelined architecture, throughput can be 1 instruction per cycle, even if the latency varies.
- Different instructions have **different pipeline depths**. **FP ALU operations** - more complex hardware and longer pipelines - may take multiple cycles (e.g., 3 cycles latency for a result to be available), but you can still start a new FP operation every cycle. Memory access delays - A **load** - might take 1 cycle before its data is ready to be used. A **store** - doesn't need the data immediately; it can proceed as soon as the address and data are ready, sometimes with 0 latency.

Instruction producing result	Instruction using result	Latency in clock cycles
FP ALU op	Another FP ALU op	3
FP ALU op	Store double	2
Load double	FP ALU op	1
Load double	Store double	0

Padlet Questions

UNIVERSITY of York

S2 Week 5 L8 - Compiler techniques 1: Latency and Clock Cycles

Why do different instructions have different latencies?

- **Pipeline hazards.** Some instructions depend on results from previous instructions. If an instruction needs data that isn't ready yet, it must wait—this is where latency matters.
- **Dependencies**. **Pipeline scheduling** and **compiler optimisations** (as in slide 8: separating dependent instructions) aim to fill these waiting gaps to minimize stalls.

Instruction producing result	Instruction using result	Latency in clock cycles
FP ALU op	Another FP ALU op	3
FP ALU op	Store double	2
Load double	FP ALU op	1
Load double	Store double	0

Padlet Questions

S2 Week 5 L8 - Compiler techniques 1: Name Dependencies (WAR & WAW)

Did I understand correctly that both of these can just be thought of as name dependencies and thus the hazards can be resolved by renaming the register (assuming of course there are enough spare registers)

Both Write After Read (**WAR**) and Write After Write (**WAW**) hazards are examples of **name dependencies**. They arise not because of a true data dependency, but because instructions use the same register name, which creates potential conflicts in the pipeline. There is no flow of data between the instructions associated with that name.

WAR hazard happens when Instruction 2 writes to a register before Instruction 1 has read it. WAW happens when two instructions write to the same register, and the writes get reordered incorrectly. Name dependency because the instructions do not actually share a data dependency—just a name conflict.

By renaming registers, the hardware or compiler can eliminate these dependencies.

Padlet Questions

S2 Week 5 L8 - Compiler techniques 2: Strip Mining

In the lecture you're talking about the need to establish a process for when the number of iterations is not known. If the number of iterations is not known, where do we get the values for n and k from? Is it more just in case you have a loop that runs, for example, an odd number of times so can't be split evenly?

Strip mining is a compiler optimisation technique used to break up loops into chunks (or "strips") of a fixed size, which makes them easier to vectorize or parallelise.

Typically, we split a loop that runs n iterations into strips of size k. So we end up with:

- A **main loop** that processes chunks of size k.
- A **remainder loop** that handles any leftover iterations if n is not a multiple of k.

Padlet Questions

S2 Week 5 L8 - Compiler techniques 2: Strip Mining

In the lecture you're talking about the need to establish a process for when the number of iterations is not known. If the number of iterations is not known, where do we get the values for n and k from? Is it more just in case you have a loop that runs, for example, an odd number of times so can't be split evenly?

Where do n and k come from?

- n is typically known at runtime, or it could be statically known if it's hardcoded in the loop.
- k is chosen by the compiler or the programmer, often based on the **vector width** of the hardware (e.g., SIMD units process 4, 8, or more elements at a time), **the cache size** or **memory hierarchy** (to optimise for locality).

Padlet Questions

S2 Week 5 L8 - Compiler techniques 2: Strip Mining

In the lecture you're talking about the need to establish a process for when the number of iterations is not known. If the number of iterations is not known, where do we get the values for n and k from? Is it more just in case you have a loop that runs, for example, an odd number of times so can't be split evenly?

What if the number of iterations isn't known ahead of time?

- If we don't know n until runtime (e.g., data-dependent loops), the code typically: Calculates the number of full strips: **n / k**; Handles the remainder with **n % k**.
- The loop is structured to process as many full chunks as possible, and then finish off any leftovers.

The main loop can run efficiently, often using vectorized instructions or parallel threads. The remainder loop ensures correctness when n isn't a perfect multiple of k (e.g., odd numbers of iterations).

Padlet Questions

S2 Week 5 L8 - Compiler techniques 2: Loop unrolling

I don't quite understand how by using different registers will help avoid unnesscessary constraints.

Loop unrolling is a compiler optimisation technique where you **repeat the loop body multiple times inside the loop** to reduce overhead and increase instruction-level parallelism (ILP).

Example (Unroll factor 2):

```
// Original
for (i = 0; i < n; i++) {
    A[i] = B[i] + C[i];
}

// Unrolled
for (i = 0; i < n; i += 2) {
    A[i]   = B[i]   + C[i];
    A[i+1] = B[i+1] + C[i+1];
}
```

Padlet Questions

S2 Week 5 L8 - Compiler techniques 2: Loop unrolling

I don't quite understand how by using different registers will help avoid unnesscessary constraints.

When we unroll a loop, we are executing more instructions in parallel or close together. If we use the same registers for each iteration, we'll likely create false dependencies: Write-after-Read (WAR) and Write-after-Write (WAW) hazards. These dependencies force instructions to wait unnecessarily, limiting parallel execution.

Example: If we unroll but reuse the same registers,

```
Loop:
    LOAD R1, B[i]
    ADD  R1, C[i]
    STORE R1, A[i]

    LOAD R1, B[i+1]   ; Reusing R1
    ADD  R1, C[i+1]
    STORE R1, A[i+1]
```

The second iteration **must** wait until the first one is done with **R1.**

Padlet Questions

S2 Week 5 L8 - Compiler techniques 2: Loop unrolling

I don't quite understand how by using different registers will help avoid unnesscessary constraints.

By using **different registers** for each iteration, we can **remove** those unnecessary constraints:

```
Loop:
   LOAD R1, B[i]
   ADD  R1, C[i]
   STORE R1, A[i]

   LOAD R2, B[i+1]   ; Using R2 instead of reusing R1
   ADD  R2, C[i+1]
   STORE R2, A[i+1]
```

Now the CPU can **execute these instructions in parallel** (or out-of-order), because there's **no false dependency** between instructions that use **different registers**.

Padlet Questions

***S2 Week 6 L9 - Dynamic scheduling 1**: "Simple Pipeline"*

Please can you define "simple pipeline"? The lecture uses this term but only infers its meaning.

A **simple pipeline** refers to a **basic implementation of instruction pipelining** in a processor. It is often characterised by:

- **In-order instruction issue**: Instructions are fetched and issued **in the exact order** they appear in the program.
- **In-order execution**: Instructions **progress through each pipeline stage in order**, without bypassing or reordering.
- **In-order completion**: Instructions **write their results in order**, typically at the end of the pipeline.

Padlet Questions

***S2 Week 6 L9 - Dynamic scheduling 1**: "Simple Pipeline"*

Please can you define "simple pipeline"? The lecture uses this term but only infers its meaning.

If an instruction stalls (e.g., waiting for data or a functional unit), all subsequent instructions must wait, even if they are independent.

There is no dynamic scheduling, no out-of-order execution, and no register renaming in a simple pipeline.

Hazards (data, control, structural) are typically managed by stalling the pipeline, not by clever avoidance techniques.

So it makes sense why more advanced techniques like: Out-of-order execution Register renaming Dynamic scheduling Speculative execution, etc.

Padlet Questions

***S2 Week 6 L9 - Dynamic scheduling 1**: "Division Cycles"*

We have seen that division will take close to 24 cycles.

We mentioned this during the previous discussion on **floating point operations**, particularly when highlighting the limitations of simple pipelining. In that context, we discussed how certain operations—like **floating point division**—can be significantly **more time-consuming** than others.

Specifically, division often requires **many more cycles** because of the complexity of the operation. While addition or subtraction on floating point numbers might complete in just a few cycles, division typically takes much longer. The figure of 24 cycles was given as an **illustrative example** to demonstrate this latency—though the exact number can vary depending on the architecture.

In many classic examples (like MIPS pipelines discussed in Hennessy and Patterson), floating point divide instructions are known to take anywhere **from 20 to 30 cycles**.

Discussion Forum

Discussion Forum - Key Themes for Discussion

1. Understanding RISC Pipelining and Its Benefits

Differences between RISC and CISC pipelines.

Key stages in a RISC pipeline and how instruction pipelining improves performance.

2. Challenges and Limitations of Pipelining

Factors that reduce pipeline speedup (hazards, stalls, branch mispredictions).

Difficulties in implementation (handling dependencies, ensuring uniform instruction execution).

Understanding latency vs. throughput—why pipelining improves throughput but not single-task latency.

Discussion Forum - Key Themes for Discussion

UNIVERSITY of York

3. Real-World Pipelining Considerations

Issues like non-uniform instruction execution causing pipeline stalls.

The ideal vs. real-world pipeline performance—how theoretical speedup differs from practical execution.

How modern architectures optimize and mitigate pipelining inefficiencies (out-of-order execution, speculation).

Discussion Forum - RISC Pipeline

Pick one or more of the following questions, discuss with your small group and add comments to the Padlet discussion board:

1. How does the RISC pipeline differ from the CISC pipeline?
2. Can you name some examples of modern processors that utilize RISC pipeline architecture?
3. What are the main stages in a RISC pipeline? What are the stages used for?
4. A number of pipe stages will contribute to potential speedup. What are the factors that might reduce the speedup or effectiveness in pipelined system?

Discussion Forum - RISC Pipeline

Pick one or more of the following questions, discuss with your small group and add comments to the Padlet discussion board:

5. How does instruction pipelining improve performance in RISC architectures?

6. What makes pipelining hard to implement?

7. Does pipelining reduce the latency of a single task? Why?

8. Pipelining Issues: Can you explain the issue of uniform-latency sub-computations in pipeline design?

9. What are the ideal cases envisioned in pipelining design and what are the realities in implementation and operation?

Discussion Forum - Approach

Small Group Discussions

– Form a small group with someone nearby, or feel free to move around and join your friends

– Pick one or more applications

– 15 mins group discussion

– Add comments/summary as a group to the Padlet discussion board

https://padlet.com/pengchengliu83/week-5-discussion-forum-i0hqf2sxaj8y60mh

Discussion Forum - Introduction to RISC Pipeline

1. How does the RISC pipeline differ from the CISC pipeline?

The RISC (**Reduced Instruction Set Computing**) pipeline typically has a simpler and more streamlined design compared to the **Complex Instruction Set Computing** (CISC) pipeline. RISC architectures focus on a smaller set of simple instructions, which allows for a more regular and efficient pipeline structure.

RISC: **simplicity and efficiency** by favouring a smaller set of simple instructions with uniform formats and regular execution semantics. CISC: provide **a rich set of complex instructions** that can perform multiple operations in a single instruction; prioritize code density and ease of programming by providing high-level abstractions and built-in support for complex operations. **Design Principle.**

Discussion Forum - Introduction to RISC Pipeline

1. How does the RISC pipeline differ from the CISC pipeline?

RISC: feature a **reduced and streamlined instruction set** with a focus on basic operations, such as load, store, arithmetic, and logical operations. CISC: support a **wide range of complex instructions**, including those for string manipulation, floating-point arithmetic, and high-level control flow. **ISA.**

RISC: have a **simpler and more regular pipeline structure with fewer stages**. Common stages in a RISC pipeline include instruction fetch, instruction decode, execute, memory access, and write back. CISC: may have **more stages and greater complexity** due to the wide variety of instruction types and formats supported by CISC architectures. **Pipeline Stages.**

Discussion Forum - Introduction to RISC Pipeline

2. Can you name some examples of modern processors that utilize RISC pipeline architecture?

Modern processors such as **ARM**-based processors, **MIPS**, and **PowerPC** utilize RISC pipeline architecture. These processors are commonly found in mobile devices, embedded systems, and networking equipment due to their efficiency and performance characteristics.

Discussion Forum - Introduction to RISC Pipeline

3. What are the main stages in a RISC pipeline?

The main stages in a RISC pipeline typically include **instruction fetch**, **instruction decode**, **execute**, **memory access**, and **write back**. These stages enable the concurrent execution of multiple instructions, leading to improved performance and throughput.

Instruction Fetch (**IF**): In this stage, the processor fetches the next instruction from memory based on the current program counter (PC). The instruction is usually located in the instruction cache to minimize memory access latency.

Instruction Decode (**ID**): In the instruction decode stage, the fetched instruction is decoded to determine its opcode and operands. The processor also identifies any control signals required to execute the instruction and resolves any register dependencies.

Execute (**EX**): The execute stage is where the actual computation or operation specified by the instruction takes place. This stage may involve arithmetic or logical operations, memory accesses, or control flow calculations, depending on the type of instruction.

Discussion Forum - Introduction to RISC Pipeline

3. What are the main stages in a RISC pipeline?

Memory Access (**MEM**): If the instruction involves memory access, such as a load or store operation, the memory access stage is responsible for reading data from or writing data to memory. This stage may also include address calculation and data alignment operations.

Write Back (**WB**): In the write back stage, the results of the executed instruction are written back to the appropriate destination, such as registers or memory. For arithmetic or logical operations, the results are typically written back to the register file.

These stages represent a simplified view of the RISC pipeline and may vary depending on the specific processor architecture and pipeline design. Additionally, some RISC processors may include additional stages, such as branch prediction or hazard detection, to improve performance and efficiency.

Discussion Forum - Introduction to RISC Pipeline

4. A number of pipe stages will contribute potential speedup. What are the factors that might reduce the speedup or effectiveness in pipelined system?

Data Hazards: Situations where data dependencies between instructions create stalls in the pipeline, e.g., read-after-write (RAW) dependencies.
Control Hazards: Situations where the pipeline must be stalled or flushed due to branch instructions or other control flow changes.

Stalls or bubbles in the pipeline can occur due to various reasons, including cache misses, branch mispredictions, or other pipeline hazards. These **stall cycles** reduce the overall efficiency of the pipeline.

If the **pipeline stages are not balanced**, meaning some stages take significantly longer than others, the overall speedup can be limited by the slowest stage (pipeline stage with the longest latency).

Memory access latency, including cache misses, can introduce delays in the pipeline. If instructions are dependent on memory operations, the pipeline may experience stalls waiting for data to be fetched.

Discussion Forum - Introduction to RISC Pipeline

4. A number of pipe stages will contribute potential speedup. What are the factors that might reduce the speedup or effectiveness in pipelined system?

Long **dependency chains** in the code can limit the effectiveness of pipelining. If instructions have long data dependencies, it may not be possible to fully utilize the pipeline, leading to idle stages. In superscalar architectures where multiple instructions can be issued in a single clock cycle, dependencies on limited execution resources can introduce stalls. Resource dependencies can limit the effective parallelism achievable in each clock cycle.

Incorrectly predicted branches (i.e. misprediction) can result in flushing the pipeline and introducing stalls. Advanced branch prediction techniques are employed to minimize such mispredictions, but they may not eliminate them entirely.

….

Discussion Forum - Introduction to RISC Pipeline

5. How does instruction pipelining improve performance in RISC architectures?

Instruction pipelining allows **multiple instructions to be processed simultaneously** by breaking down the instruction execution process into sequential stages. As a result, the overall throughput of the processor is increased, and the latency of individual instructions is reduced.

In a pipelined RISC architecture, multiple instructions can be in different stages of execution simultaneously. While one instruction is being executed in a given stage, the subsequent instruction can be fetched, decoded, or have its operands prepared. This enables **concurrent execution of multiple instructions**, leading to higher throughput.

Pipelining **reduces the latency of individual instructions** by breaking down the execution process into sequential stages. Each stage performs a specific operation, such as instruction fetch, decode, execute, memory access, and write back. As a result, the time taken to complete each instruction is reduced since different instructions can progress through the pipeline simultaneously.

Discussion Forum - Introduction to RISC Pipeline

5. How does instruction pipelining improve performance in RISC architectures?

Pipelining allows for **better utilization of hardware resources**. While one stage of the pipeline is processing an instruction, other stages can be used to process subsequent instructions. This means that the hardware components, such as the arithmetic logic unit (ALU) or memory units, can be kept busy more effectively, leading to improved overall efficiency.

With pipelining, the execution of one **instruction overlaps** with the execution of subsequent instructions. This overlap minimizes idle time in the processor, making better use of available processing resources and increasing overall system performance.

Pipelining facilitates **scalability in RISC architectures**. As the number of pipeline stages increases or additional pipeline features are introduced (such as instruction-level parallelism), the performance of the processor can be further enhanced without fundamentally altering its architecture.

Discussion Forum - Introduction to RISC Pipeline

6. What makes pipelining hard to implement?

Read-After-Write (RAW) Dependencies: **Data hazards** occur when an instruction depends on the result of a previous instruction that has not yet completed. Handling dependencies and ensuring the correct flow of data through the pipeline can be challenging.

Branches & Jumps: **Control hazards** arise with branch instructions, leading to uncertainties in program flow. Accurate branch prediction and handling of mispredictions are crucial for maintaining pipeline efficiency.

Stall Handling: Pipelining introduces the potential for **pipeline stalls**, where certain stages are idle due to dependencies, cache misses, or other issues. Implementing efficient mechanisms to handle stalls and maintain pipeline throughput is **complex**.

Dependency Management: Properly **decoding and issuing instructions** while managing dependencies and ensuring in-order execution can be **complex**. Efficient instruction scheduling is crucial for maximizing pipeline utilization.

Discussion Forum - Introduction to RISC Pipeline

6. What makes pipelining hard to implement?

Hazard Resolution: Implementing **data forwarding mechanisms** to resolve hazards and allow data to bypass pipeline stages experiencing stalls requires careful design. Identifying dependencies and forwarding data correctly is essential for pipeline efficiency.

Memory Access Latency: **Handling cache misses and memory access latency** is a significant challenge. Strategies such as out-of-order execution or speculative execution may be employed to hide memory latency, but they introduce additional complexity.

Stage Latency Differences: Ensuring that all pipeline stages have similar latencies is challenging. **Imbalances in stage latencies** can lead to pipeline stalls, reducing overall performance.

Resource Dependencies: Implementing **superscalar architectures or out-of-order execution** introduces complexities related to managing dependencies on execution resources. This involves sophisticated reservation stations and issue queues.

Discussion Forum - Introduction to RISC Pipeline

UNIVERSITY of York

6. What makes pipelining hard to implement?

Interrupts and Exceptions: **Managing interrupts and exceptions** within the pipeline requires careful consideration. Flushing the pipeline and handling exceptional conditions appropriately to maintain correct program execution adds complexity.

Correctness Assurance: Pipelined architectures are complex, and **testing and validating** their correct behavior under various conditions can be challenging. Verifying the correctness of the pipeline in the presence of corner cases and rare events is essential.

Startup and Shutdown Overheads: The **time it takes to fill the pipeline initially and the time it takes to drain the pipeline** at the end of a program can contribute to inefficiencies, especially in short programs.

Discussion Forum - Introduction to RISC Pipeline

7. Does pipelining reduce the latency of a single task? Why?

While pipelining can improve **throughput of the entire workload**, it may not necessarily reduce the latency of an individual task. Pipelining can reduce the latency of a single task when the task can be broken down into multiple stages, each of which can be executed concurrently in a pipeline. The key idea behind pipelining is to **overlap the execution of multiple tasks, allowing the overall throughput of the system to be higher.**

Pipelining allows multiple tasks to be in various stages of execution **simultaneously**. As a result, the time it takes to complete a task is reduced because different stages are working concurrently on different tasks.

While one stage of the pipeline is working on one task, the next stage can start processing the next task. This **overlapping of processing stages** helps to utilize hardware resources more efficiently.

Discussion Forum - Introduction to RISC Pipeline

7. Does pipelining reduce the latency of a single task? Why?

Here's how pipelining can reduce latency:

Even though the latency of an individual task might not be reduced significantly, the **overall throughput** of the system can be increased. This is particularly beneficial in scenarios where there is a stream of tasks, and the focus is on processing them in a continuous and efficient manner.

While pipelining is designed to improve throughput and enable concurrent processing of multiple tasks, its impact on the latency of a single task depends on various factors, including the **efficiency of the pipeline design**, the **handling of potential stalls and dependencies**, and the **balance of pipeline stages**. In an ideal scenario with minimal stalls and well-balanced stages, pipelining can contribute to reducing the latency of a series of tasks.

Discussion Forum - Introduction to RISC Pipeline

8. Pipelining Issues: Can you explain the issue of uniform-latency sub-computations in pipeline design?

The concept of uniform-latency sub-computations refers to the **ideal scenario where each stage of a pipeline takes exactly the same amount of time to complete its processing**. This uniformity simplifies pipeline design, scheduling, and optimization, leading to predictable performance and efficient resource utilization. However, in practice, achieving uniform-latency sub-computations across all pipeline stages is challenging due to various factors.

In the ideal scenario, uniform-latency sub-computations ensure **predictable performance characteristics** throughout the pipeline. Each stage takes the same amount of time to process an instruction or data item, enabling straightforward performance analysis and optimization.

Uniform-latency sub-computations **simplify pipeline design** by ensuring that all pipeline stages have identical characteristics and requirements. This simplification facilitates the design, implementation, and testing of the pipeline architecture.

Discussion Forum - Introduction to RISC Pipeline

8. Pipelining Issues: Can you explain the issue of uniform-latency sub-computations in pipeline design?

With uniform-latency sub-computations, hardware resources can be evenly distributed across pipeline stages, leading to **balanced resource utilization** and efficient hardware design.

Uniform latencies **simplify instruction scheduling and dependency management** within the pipeline. Since all stages take the same amount of time to execute, scheduling decisions become more straightforward, and dependencies are easier to analyze and resolve.

However,

Discussion Forum - Introduction to RISC Pipeline

8. Pipelining Issues: Can you explain the issue of uniform-latency sub-computations in pipeline design?

In practice, **not all pipeline stages may have uniform latencies** due to differences in hardware complexity, resource contention, and processing requirements. For example, memory access and arithmetic operations may exhibit different latencies within the pipeline.

Certain pipeline stages may have **dependencies on external resources or subsystems** that introduce variable latency. For instance, memory access latency can vary depending on cache hierarchy, memory controller performance, and memory access patterns.

Structural, data, and control **hazards** can introduce variability in pipeline latency by causing stalls, bubbles, and pipeline flushes. Resolving these hazards often requires additional cycles, leading to non-uniform latency across pipeline stages.

Discussion Forum - Introduction to RISC Pipeline

8. Pipelining Issues: Can you explain the issue of uniform-latency sub-computations in pipeline design?

Variations in workload characteristics and instruction mix can affect pipeline latency. Workloads with diverse instruction types, data dependencies, and branching behavior may experience different latencies in various pipeline stages.

Process variations, technology scaling effects, and manufacturing **variability** can introduce differences in pipeline latency across different processor implementations and generations.

Despite efforts to achieve uniform-latency sub-computations, **performance tuning and optimization** may prioritize certain pipeline stages over others, leading to non-uniform latency to maximize overall system performance.

Discussion Forum - Introduction to RISC Pipeline

9. What are the ideal cases envisioned in pipelining design and what are the realities in implementation and operation?

Ideal Cases in Pipelining:

In an ideal scenario, pipelining allows for a **significant increase in throughput**, enabling multiple instructions to be processed simultaneously and efficiently.

Pipelining ideally leads to **improved performance** by reducing the time taken to execute individual instructions and increasing the overall rate of instruction execution.

Ideal pipelining designs **maximize resource utilization** by efficiently overlapping the execution of instructions across different pipeline stages, ensuring that hardware resources are fully utilized.

In ideal cases, pipelines exhibit **regular and predictable behavior**, allowing for efficient instruction scheduling and resource management.

Discussion Forum - Introduction to RISC Pipeline

9. What are the ideal cases envisioned in pipelining design and what are the realities in implementation and operation?

Realities in Pipelining:

In reality, **pipeline hazards** such as data hazards, control hazards, and structural hazards can significantly impact pipeline efficiency and performance. Handling these hazards requires additional hardware logic and software techniques, which can introduce complexity and overhead.

Pipelines often experience **stalls and bubbles** due to hazards, resource contention, or pipeline flushes. These stalls reduce throughput and diminish the benefits of pipelining.

Longer pipelines, while potentially offering greater throughput, can introduce higher latency and overhead due to increased pipeline depth. Balancing **pipeline length** with performance considerations is crucial in practice.

Discussion Forum - Introduction to RISC Pipeline

9. What are the ideal cases envisioned in pipelining design and what are the realities in implementation and operation?

Realities in Pipelining:

Managing dependencies between instructions **introduces overhead** in terms of forwarding logic, bypassing mechanisms, and branch prediction. This overhead can limit the achievable performance gains from pipelining.

In the event of mispredicted branches or other exceptions, **pipelines may need to be flushed**, discarding partially executed instructions and introducing latency and performance penalties.

Coordinating and synchronizing the operation of multiple pipeline stages and units can **introduce complexity and overhead**, particularly in multi-core or multi-threaded architectures.

Pipelining can lead to **increased power consumption** and **energy usage**, particularly in high-performance computing systems. Managing power and thermal considerations becomes essential in modern architectures.

Closing Remarks and Announcement

- **Next Week (Week 7)**:

Lecture: In Week 7, our discussion will be focused on some principles of speculative execution and superscalar processors in Lecture 11 and Lecture 12, respectively.

Lab: We will continue working on solutions for Week6's lab tasks.

Exercises/Quizzes: Solution of Exercise 2 will be released on 27/03/2026. Quiz 2 will be discussed in the Lecture Review session on 26/03/2026.

Thanks

See You Next Week !